



جامعة سيئون
كلية الحاسبات

ملزمة جامعية · البرمجة كائنية التوجه

التجريد في البرمجة
(Abstraction)

مفهوم Abstraction | الكلمة abstract | Abstract Class | Abstract Methods

Concrete Methods | الوراثة من Abstract Class | قيود إنشاء Objects | تطبيق التجريد

إعداد الطالب
حسين جمال الليل بن عقيل

لغة الأمثلة: C#
بيئة العمل: Visual Studio

محتويات الملزمة
Contents

1	مفهوم التجريد وأهميته	Abstraction	3
2	الكلمة abstract و Abstract Class	abstract keyword	4
3	الدوال المجردة	Abstract Methods	5
4	الدوال المكتملة	Concrete Methods	6
5	الوراثة من Abstract Class	Inheritance	7
6	قيود إنشاء Objects من Abstract Class	Instantiation Rules	9
7	تطبيق التجريد في تصميم البرامج	Full Example	10
8	الملخص	Summary	11
9	تمرين تطبيقي ومساحة الحل	Exercise	12

أهداف الملزمة

بعد ما تخلّص هذي الملزمة، المفروض تكون قادر على:

1. تشرح فكرة التجريد بكلامك وبمثال من الحياة.
2. تكتب Abstract Class بلغة C#.
3. تفرّق بين Abstract Method و Concrete Method.
4. ترث من كلاس مجرد وتكمّل دواله بـ `override`.
5. تعرف ليش ما نقدر نسوي `new` لكلاس مجرد.
6. تطبّق التجريد في برنامج صغير متكامل.

> **ملاحظة حول الأمثلة**
>
> كل الأكواد مكتوبة بلغة C# ومجرَّبة، ومعروضة كلقطات من Visual Studio. نمشي على مثال واحد يكبر معنا خطوة بخطوة: **الأشكال الهندسية** (Shape ← Circle و Rectangle).

1 مفهوم التجريد
What is Abstraction?

التجريد (Abstraction) هو أننا **نُظهر المهم فقط** للي يستخدم الشيء، و**نخفي التفاصيل الداخلية** المعقدة. يعني نركّز على **"ماذا يعمل؟"** بدل **"كيف يعمل من الداخل؟"**.

بالعامي

أنت لما تسوق السيارة، تستخدم الدركسون والفرامل والبنزين وبس. ما يهمك كيف المحرك يحرق الوقود ولا كيف الجير يتنقل. السيارة "جرّدت" لك الموضوع وعطتك اللي تحتاجه بس.

مثال السيارة كمخطط



السهم المتقطع = جزء مخفي عن المستخدم

وفي البرمجة؟

نفس الفكرة بالضبط: نكتب **كلاس عام** يحدد **"إيش لازم يكون موجود"** (مثلاً: كل شكل لازم له مساحة)، ونترك **"كيف تنحسب"** لكل كلاس فرعي حسب نوعه. الدائرة تحسبها بطريقة، والمستطيل بطريقة ثانية.

ليش التجريد مهم؟

- **يبسّط البرنامج:** اللي يستخدم الكلاس يشوف شي بسيط وواضح.
- **يوحّد الشكل:** كل الكلاسات الفرعية مُلزمة بنفس الدوال، فما أحد ينسى شي مهم.
- **يسهّل التعديل والتوسعة:** تضيف شكل جديد (مثل مثلث) بدون ما تلمس الكود القديم.
- **يقلل الأخطاء:** التفاصيل مخفية، فما أحد يعبث فيها بالغلط.

التعريف باختصار

التجريد = إخفاء التفاصيل + إظهار الأساسيات. وفي C# نطبّقه بشكل رئيسي عن طريق **Abstract Class** و **Interface**، وفي هذي الملزمة نركّز على Abstract Class.

2 الكلمة abstract و Abstract Class

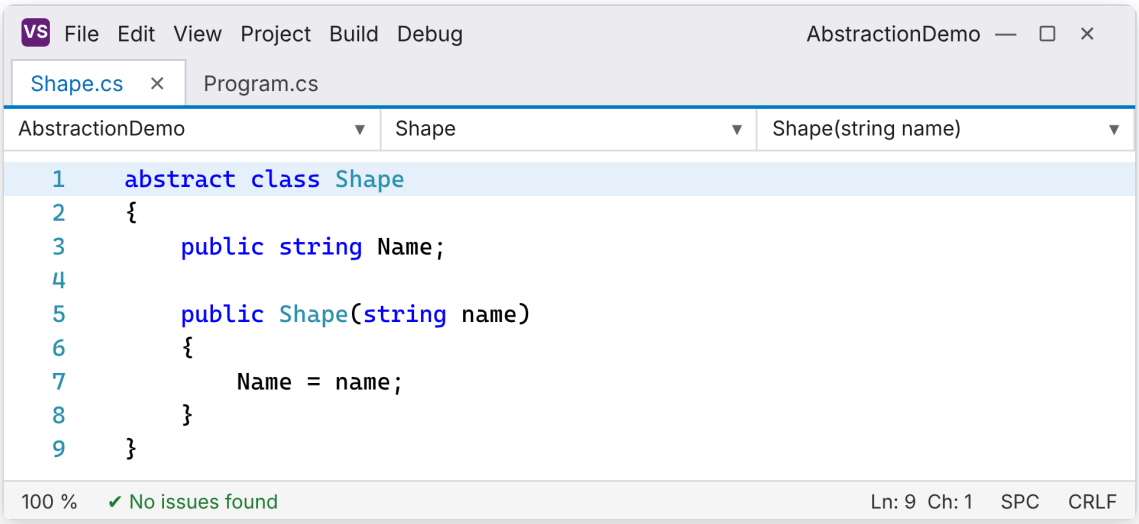
The abstract keyword

الكلمة المحجوزة `abstract` معناها **"ناقص / غير مكتمل"**. لما نحطها قبل `class` يصير عندنا **Abstract Class**: كلاس يمثّل **فكرة عامة**، ويُستخدم **كأساس** لكلاسات ثانية ترث منه.

بالعامي

كلمة "شكل" لحالها فكرة عامة. ما في شي اسمه "شكل" وبس! في دائرة، في مستطيل، في مثلث. فـ Shape هو "القالب العام"، والأشكال الحقيقية هي اللي تطلع منه.

طريقة الكتابة



كود (1): تعريف كلاس مجرد بسيط باسم Shape

شرح الكود

1 كلمة `abstract` قبل `class` هي اللي تخلي Shape كلاس مجرد.

3 عادي يكون فيه متغيرات (Fields) مثل `Name`.

5–8 وعادي يكون فيه Constructor، يشتغل لما ننشئ كائن من كلاس **فرعي** يرث منه.

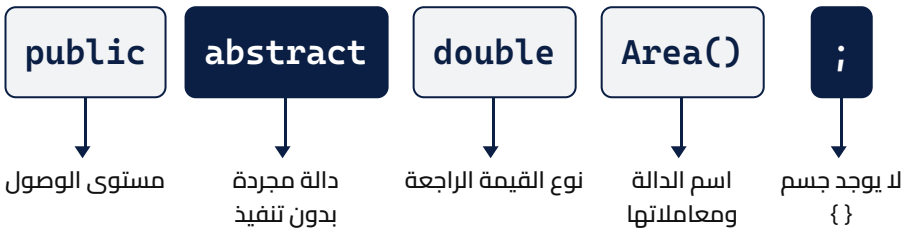
قاعدة مهمة

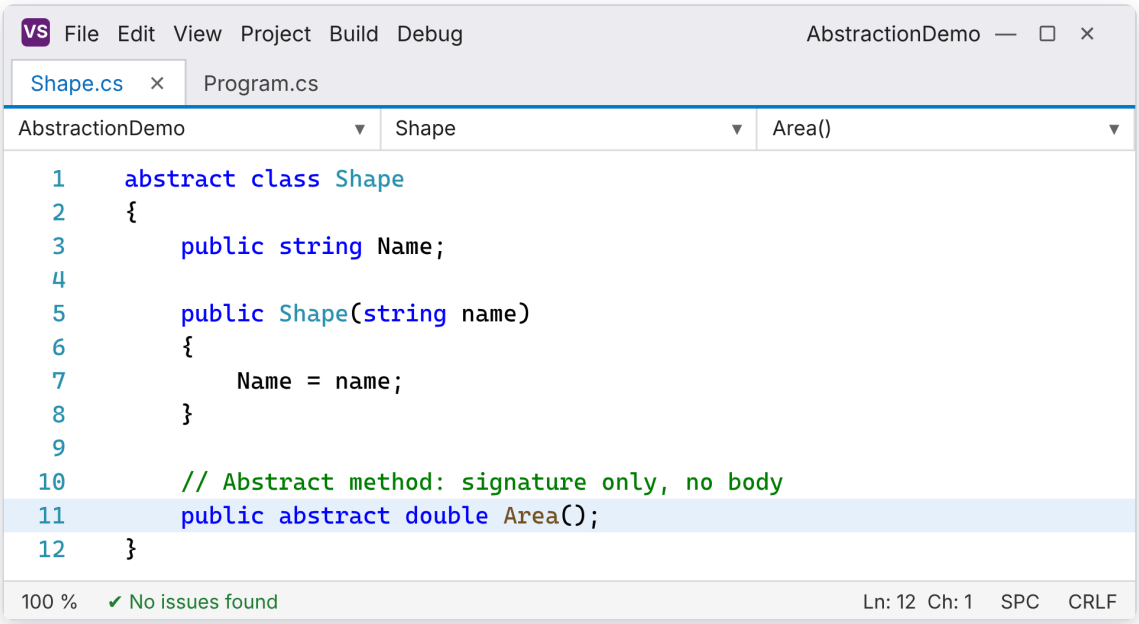
الـ Abstract Class كلاس طبيعي في كل شي تقريبًا (متغيرات، Constructor، دوال)، **الفرق الوحيد** أنه **ما ينفع ننشئ منه كائن مباشرة**، وممكن يحتوي على دوال مجردة (نشوفها في الصفحة الجاية).

3 الدوال المجردة
Abstract Methods

Abstract Method هي دالة **نكتب توقيعها فقط** (الاسم ونوع الإرجاع والمعاملات) **بدون جسم** `{ }`، وننهيها بفاصلة منقوطة `;`. هي عبارة عن **"وعد"**: كل كلاس يرث لازم يكتب جسمها بنفسه.

تشريح السطر





كود (2): إضافة دالة مجردة Area() إلى Shape

شرح الكود

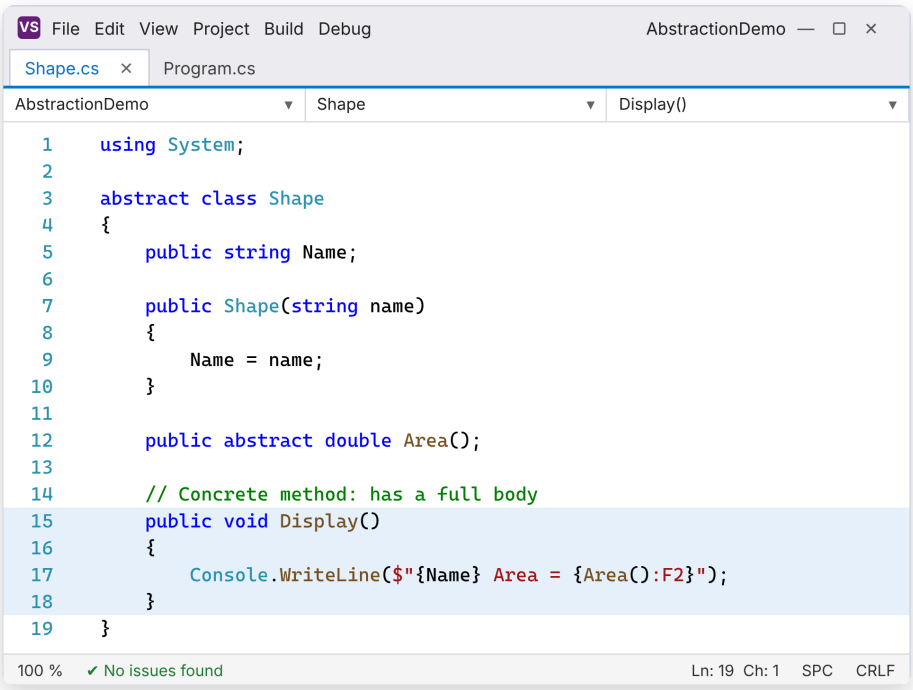
11 `public abstract double Area();` ← نقول: "كل شكل **لازم** يعرف يحسب مساحته"، بس ما نقول **كيف**؛ لأن طريقة الحساب تختلف من شكل لشكل.

شروط الدالة المجردة

- ما تنكتب إلا **داخل Abstract Class**. لو حطيتها في كلاس عادي يطلع خطأ.
- **ما لها جسم أبدًا**؛ لو كتبت لها `{ }` يعطيك المترجم خطأ.
- الكلاس الفرعي (غير المجرد) **مُلزَم** يكتب جسمها باستخدام `override`.

4 الدوال المكتملة
Concrete Methods

Concrete Method هي دالة **عادية لها جسم كامل**. الـ Abstract Class يقدر يجمع النوعين: دوال مجردة (يكملها الأبناء) + دوال مكتملة (يستخدمها الأبناء جاهزة بدون ما يعيدوا كتابتها).



كود (3): دالة مكتملة Display() داخل الكلاس المجرد

شرح الكود

1 `using System;` عشان نستخدم `Console`.

12 `Area()` مجردة: كل شكل يحسبها بطريقته.

15–18 `Display()` مكتملة: تطبع اسم الشكل ومساحته. لاحظ إنها **تنادي** `Area()` مع إنها مجردة! وقت التشغيل تتنفذ نسخة الكلاس الحقيقي (دائرة أو مستطيل).

17 `{Area():F2}` تعني: اطبع المساحة برقمين بعد الفاصلة.

مقارنة سريعة

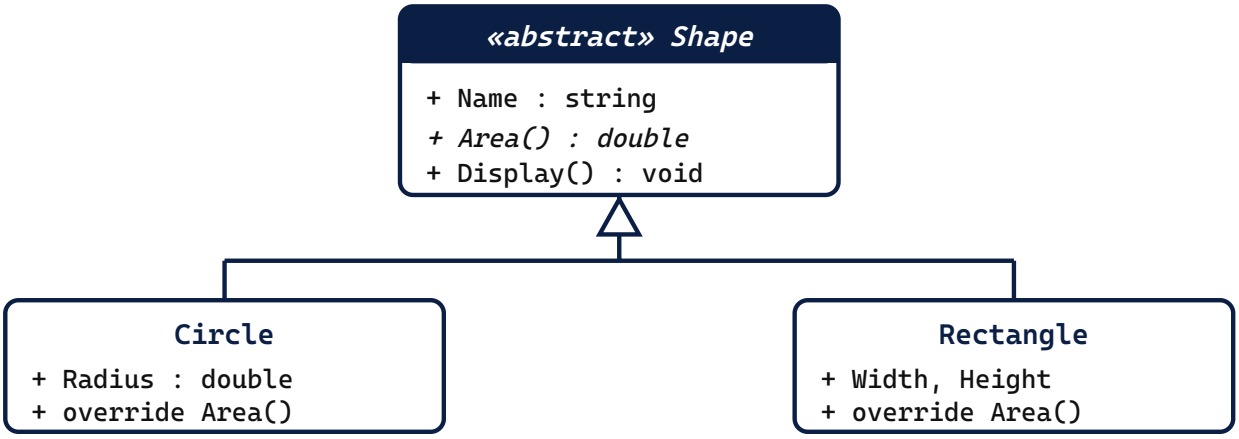
وجه المقارنة	Abstract Method	Concrete Method
الجسم { }	لا يوجد، تنتهي بـ `;`	موجود وكامل
الكلمة المميزة	`abstract`	لا شيء (دالة عادية)
مكانها	داخل Abstract Class فقط	أي كلاس
في الكلاس الفرعي	إلزامي تكتبها بـ `override`	تُورث وتُستخدم جاهزة

5 الوراثة من Abstract Class

Inheriting from an Abstract Class

الـ Abstract Class ما له فايدة لحاله؛ فايدته لما **نورّث منه**. في C# نكتب النقطتين `:` بعد اسم الكلاس الفرعي، ثم نكمّل كل دالة مجردة باستخدام `override` .

مخطط الوراثة



السهم ذو الرأس المثلث يعني "يرث من" · الخط المائل في Area() يعني دالة مجردة

```
File  Edit  View  Project  Build  Debug                    AbstractionDemo
Circle.cs ×   Shape.cs   Program.cs
AbstractionDemo ▾   Circle ▾   Area() ▾
 1   class Circle : Shape
 2   {
 3       public double Radius;
 4
 5       public Circle(double radius) : base("Circle")
 6       {
 7           Radius = radius;
 8       }
 9
10       public override double Area()
11       {
12           return Math.PI * Radius * Radius;
13       }
14   }
100 %   ✔ No issues found                  Ln: 14  Ch: 1   SPC   CRLF
```

كود (4): الكلاس Circle يرث من Shape

شرح الكود

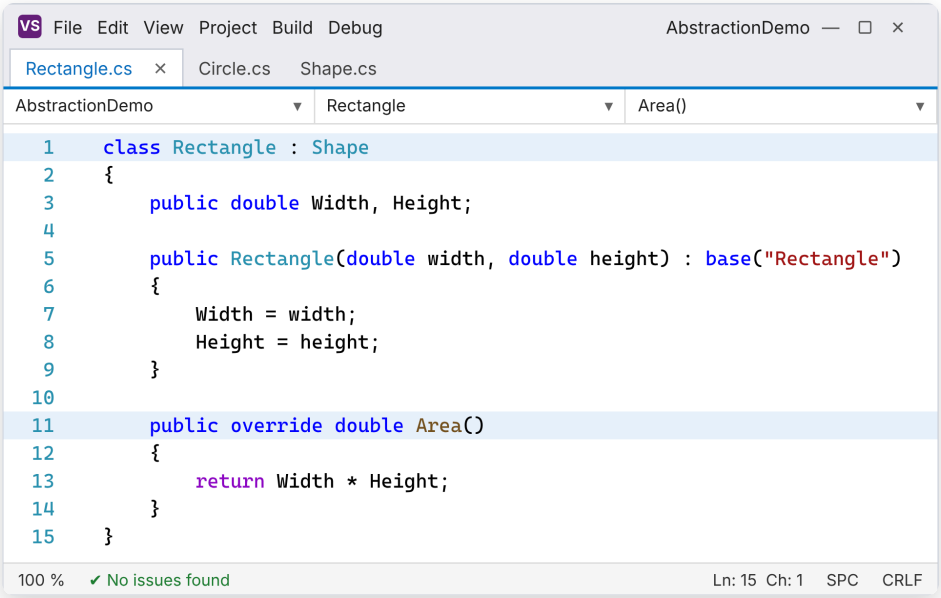
1 `Circle : Shape` ← الدائرة ترث كل شي من Shape: `Name` و `Display()` .

5 `: base("Circle")` ← تنادي Constructor الأب وتعطيه اسم الشكل.

10–13 `override` ← هنا نكتب "الطريقة" اللي وعدنا فيها: مساحة الدائرة $\pi \times r^2$.

كلاس ثاني يرث من نفس الأب

بنفس الطريقة نكتب المستطيل. لاحظ: **نفس الدالة** `Area()` لكن **بتنفيذ مختلف**. هذا هو جوهر التجريد.



كود (5): الكلاس Rectangle يرث من Shape

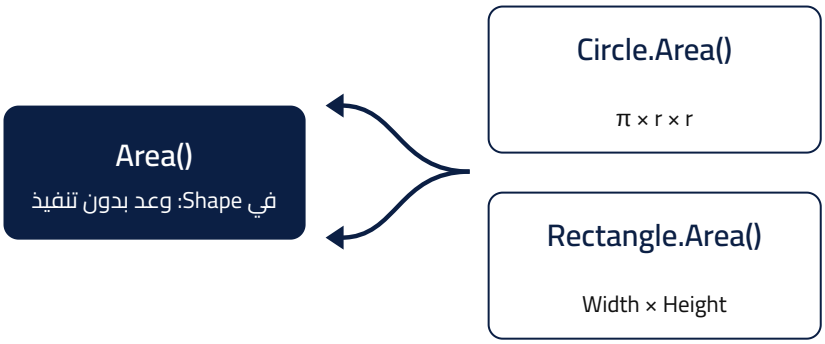
شرح الكود

1 `Rectangle : Shape` ← المستطيل يرث من نفس الأب.

5–9 الـ Constructor يستقبل العرض والطول ويرسل الاسم `"Rectangle"` للأب.

11–14 مساحة المستطيل = العرض × الطول.

نفس الطلب.. وتنفيذ مختلف



انتبه: ماذا لو نسيت override؟

لو كتبت `class Circle : Shape` وما كتبت دالة `Area()`، المترجم يرفض ويعطيك الخطأ `CS0534`:

```
'Circle' does not implement inherited abstract member 'Shape.Area()'
```

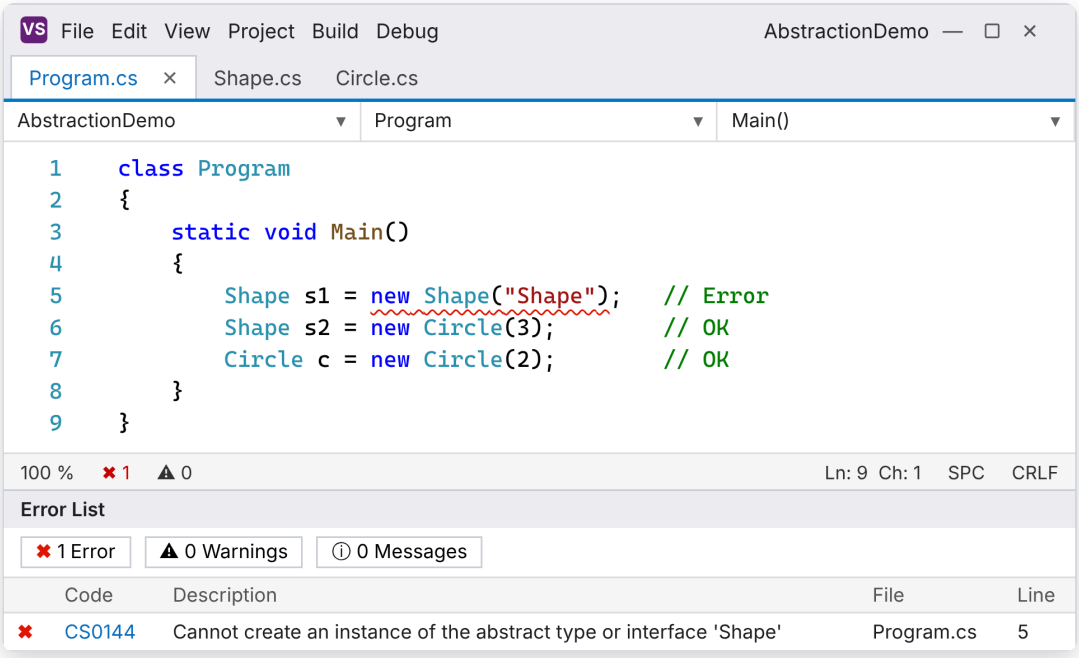
الحل: إما تكتب الدالة بـ `override`، أو تخلي Circle نفسه `abstract`.

6 قيود إنشاء Objects من Abstract Class
Instantiation Rules

ما نقدر ننشئ كائن من Abstract Class باستخدام `new`؛ لأنه ناقص (فيه دوال بدون تنفيذ). لكن **نقدر** نستخدمه **كنوع للمتغير** ونخزّن فيه كائن من كلاس فرعي.

بالعامي

لو قلت للنجار "سوّ لي **أثاث**"، بيسألك: "أي أثاث؟ كرسي ولا طاولة؟". ما يقدر يصنع "أثاث" بشكل عام! بس يقدر يصنع كرسي، والكرسي **هو** أثاث.



كود (6): المحاولة الخاطئة والمحاولات الصحيحة، مع رسالة الخطأ في Error List

شرح الكود

5 `new Shape(...)` ✗ خطأ `CS0144`: ما يمكن إنشاء كائن من نوع مجرد (الخط الأحمر المتعرّج).

6 `Shape s2 = new Circle(3);` ✓ صحيح: المتغير نوعه Shape لكن الكائن الحقيقي دائرة.

7 `Circle c = new Circle(2);` ✓ صحيح: كلاس عادي مكتمل.

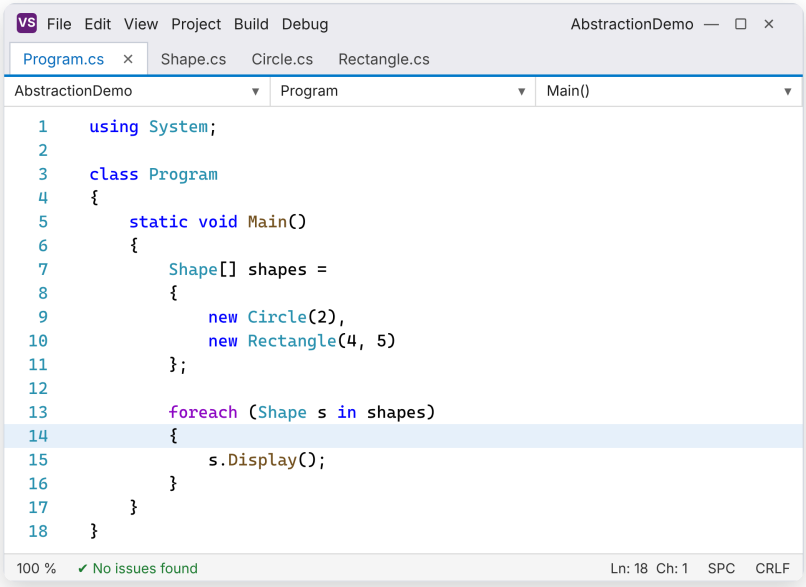
كيف يبدو السطر 6 في الذاكرة؟



7 تطبيق التجريد في تصميم البرامج

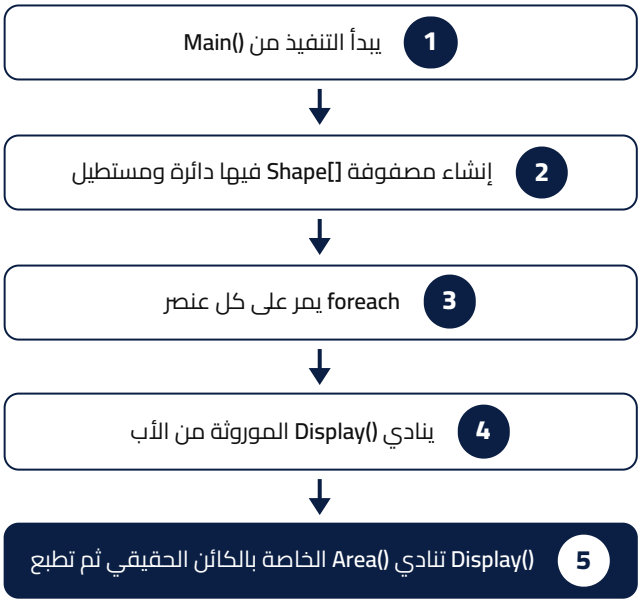
Putting it all together

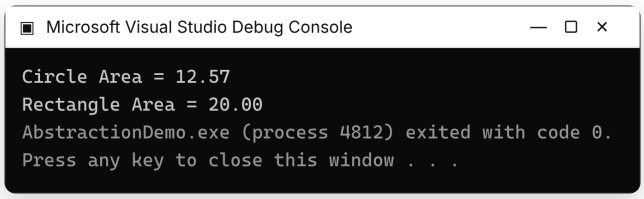
الآن نجمع كل شي: مصفوفة واحدة من نوع Shape فيها أشكال مختلفة، ونتعامل معها كلها **بنفس الطريقة** بدون ما نهتم بنوع كل شكل.



كود (7): البرنامج الرئيسي

تسلسل التنفيذ





ناتج التشغيل في نافذة Debug Console

شرح الكود والناتج

7–11 المصفوفة نوعها Shape، ونقدر نحط فيها أي كلاس فرعي منه.

13–16 نفس السطر `s.Display()` يطلع نتيجة مختلفة حسب الكائن: الدائرة $\pi \times 2^2 \approx 12.57$ والمستطيل $4 \times 5 = 20.00$.

لو بكرة أضفنا Triangle، ما نحتاج نغيّر سطر واحد في الحلقة. **هذي قوة التجريد.**

8 الملخص
Summary

مفهوم Abstraction
نُظهر "ماذا" ونخفي "كيف". نركّز على الأساسيات ونخبّي التفاصيل المعقدة.

الكلمة abstract
تعني "ناقص". تنكتب قبل class أو قبل الدالة.

Abstract Class
كلاس عام يكون أساس لغيره. فيه متغيرات و Constructor ودوال مجردة ومكتملة.

Abstract Method
توقيع فقط بدون جسم، ينتهي بـ `;` . الكلاس الفرعي مُلزَم يكملها.

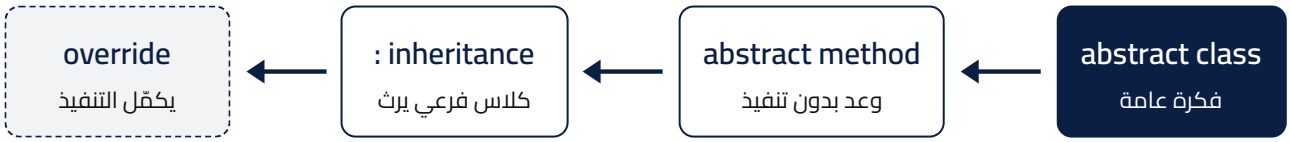
Concrete Method
دالة عادية لها جسم، تُورث وتُستخدم جاهزة.

الوراثة
`class Circle : Shape` ثم تكملة الدوال المجردة بـ `override` .

قيود إنشاء Objects
`new Shape()` ✗ خطأ، لكن
`Shape s = new Circle()` ✓ صحيح.

تطبيق التجريد
نتعامل مع كائنات مختلفة بنفس الكود، ونضيف أنواع جديدة بسهولة.

الخريطة الذهنية في سطر واحد



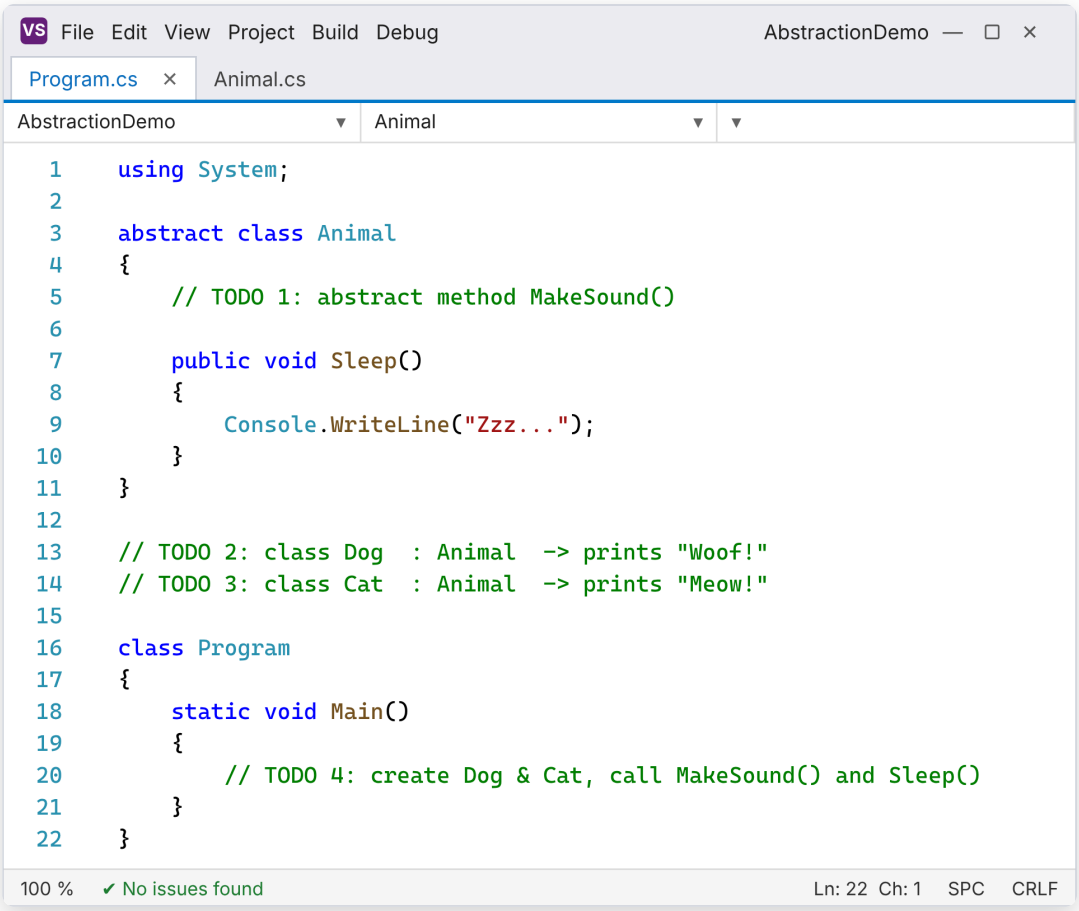
جملة وحدة تحفظها
"الـ Abstract Class يقول **إيش** لازم يصير، والكلاس الفرعي يقول **كيف** يصير"

9 تمرين تطبيقي

نحلّه سوا · Exercise

المطلوب: أكمل الكود التالي بحيث:

- يحتوي الكلاس المجرد Animal على دالة مجردة `MakeSound()` لا ترجع قيمة.
- ينشئ كلاسين Dog و Cat يرثان من Animal، كل واحد يطبع صوته.
- في `Main()` أنشئ كلبًا وقطة، وناد `MakeSound()` لكل واحد، ثم `Sleep()` للكلب.



الكود المبدئي للتمرين

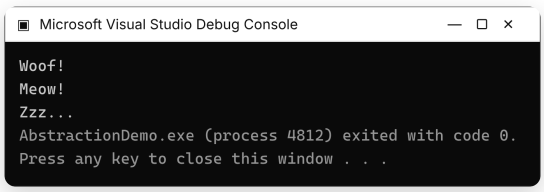
تلميح

الدالة المجردة في Animal:

```
public abstract void MakeSound();
```

وفي Dog و Cat:

```
public override void MakeSound()
```



الناتج المتوقع

مساحة الحل

اكتب الحل هنا خطوة بخطوة مع الطلاب: **1** الدالة المجردة ← **2** الكلاس Dog ← **3** الكلاس Cat ← **4** الدالة ()Main.